



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统设计实验报告

PA 1 实验报告

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：关浩

2026 年 4 月 29 日

目录

一、 必答题	2
(一) 必答题 1: i386 手册查阅	2
(二) 必答题 2: 代码量统计	4
(三) 必答题 3: gcc 编译选项	8
二、 文内思考题	9
(一) Q: cpu_exec() 传参	9
(二) Q: static 关键字的作用	9
(三) Q: int3 相关问题	10
(四) Q: 断点的设置位置	10
(五) Q: 模拟器和调试器的区别	10
三、 代码实现	11
(一) TODO: 实现正确的寄存器结构体	11
(二) TODO: 添加单步执行命令 <code>si</code>	12
(三) TODO: 添加寄存器打印命令 <code>info r</code>	14
(四) TODO: 添加扫描内存命令 <code>x N EXPR</code> (简化版)	15
(五) CHECKPOINT: PA1 stage1	16
(六) TODO: 算术表达式的词法分析	17
(七) TODO: 实现算数表达式递归求值 (含负号)	19
(八) TODO: 实现更完整的表达式求值	24
(九) TODO: 实现 <code>p</code> 和 <code>x</code> 命令	27
(十) CHECKPOINT: PA1 stage2	32
(十一) TODO: 实现监视点池的管理	32
(十二) TODO: 实现监视点实现监视点设置命令 <code>w</code> 删除命令 <code>d</code>	34
(十三) CHECKPOINT: PA1 stage3	39
四、 实验环境说明	41
(一) 虚拟机配置	41
(二) IDE 配置	41
五、 总结	42

一、 必答题

(一) 必答题 1: i386 手册查阅

问：查阅 i386 手册理解了科学查阅手册的方法之后，请你尝试在 i386 手册中查阅以下问题所在的位置，把需要阅读的范围写到你的实验报告里面：

- EFLAGS 寄存器中的 CF 位是什么意思？
- ModR/M 字节是什么？
- mov 指令的具体格式是怎么样？

答：

回答本题时参考来自 [NJU GitHub 仓库](#) 内提供的 [在线阅读页面](#)，选择它是因为它来自于 NJU 的官方 GitHub 仓库，而本实验是来自于 NJU 的，适配性较好；而且这个页面版本的 manual 支持很多文内跳转，十分方便。

(1) EFLAGS 寄存器中的 CF 位是什么意思？

查阅路径：[Chapter 2 – Basic Programming Model -> 2.3 Registers -> 2.3.4 Flags Register -> 2.3.4.1 Status Flags -> Appendix C](#)

答案范围：[Appendix C](#)

2.3.4 这一节专门介绍了 i386 中的 32 位标志寄存器 EFLAGS，它将 EFLAGS 的 FLAGS 位分为了 3 类 – “the status flags, the control flags, and the systems flags.” CF 属于 status flags.

在 2.3.4.1 节中手册写道：“The status flags of the EFLAGS register allow the results of one instruction to influence later instructions. The arithmetic instructions use OF, SF, ZF, AF, PF, and CF. The SCAS (Scan String), CMPS (Compare String), and LOOP instructions use ZF to signal that their operations are complete. There are instructions to set, clear, and complement CF before execution of an arithmetic instruction. Refer to Appendix C for definition of each status flag.” 关键句是：“The arithmetic instructions use OF, SF, ZF, AF, PF, and CF.” – 算术指令会使用到 OF、SF、ZF、AF、PF 和 CF 这些标志位。“There are instructions to set, clear, and complement CF before execution of an arithmetic instruction.” – 算术指令执行之前，还有一些指令可以对 CF 进行置位、清零或取反操作。“Refer to Appendix C for definition of each status flag.” 关于每个状态标志的定义，请参考附录 C。——这里的信息告诉了我们与 CF 有关的指令类型，以及其他一些指令相关的 CF 位的信息，同时也告诉我们要去附录 C 进一步查阅 CF 标志位的具体含义。

在[附录 C](#) (“Status Flag Summary”) 中，页面头部就是 “Status Flags’ Functions”，其中有对 CF 位的定义：

Bit	Name	Function
0	CF	Carry Flag -- Set on high-order bit carry or borrow; cleared otherwise.

(剩余表略)

上面语句原意是：CF - 进位标志 - 当最高位发生进位或借位时置位；否则清零。

这就回答了问题：CF 就是在算术运算中，表示是否发生了最高位的进位或借位的标志位。

(2) ModR/M 字节是什么?

查阅路径: [Chapter 17 – 80386 Instruction Set -> 17.2 Instruction Format -> 17.2.1](#)

ModR/M and SIB Bytes

答案范围: [17.2.1 ModR/M and SIB Bytes](#)

称之为“字节”，说明可能是指令的一部分。i386 manual 的 Chap. 17 是专门介绍指令集构成的，17.2 节介绍格式，应该能找到对这个“字节/字段”的相关定义等信息。在 17.2.1 节找到了 ModR/M 的信息：“The ModR/M and SIB bytes follow the opcode byte(s) in many of the 80386 instructions. They contain the following information: ... (略)” “The ModR/M byte contains three fields of information: ... (略)”——很清晰，这是有关 ModR/M “是什么”的介绍。原文较长，此处不展示，大致的总结如下：

总得来说，ModR/M 字节和 SIB 字节包含指令中要使用的寻址类型或寄存器编号、要使用的寄存器或用于进一步选择具体指令的信息、基址 (base)、变址 (index) 以及比例因子 (scale) 等信息。

ModR/M 有三个字段：

- **mod 字段** (最高 2 位)：与 r/m 字段结合形成 32 种取值，包括 8 种寄存器寻址和 24 种索引 (寻址) 模式。
- **reg 字段** (往后 3 位)：指定一个寄存器编号，或提供额外 3 位操作码信息——由指令的 opcode 字节决定。
- **r/m 字段** (最低 3 位)：与 mod 字段结合作为寻址模式编码的一部分 (即 m)，或指定操作数所在的寄存器 (即 r)。

对问题的简要回答: ModR/M 是 x86 指令中的一个字节，用来编码操作数的位置 (寄存器或内存) 以及具体寻址方式。

(3) mov 指令的具体格式是怎么样的?

查阅路径: [Chapter 3 – Applications Instruction Set -> 3.1 Data Movement Instructions](#)
-> 文中的 **MOV** 超链接

备选路径: [Chapter 17 – 80386 Instruction Set -> MOV Move Data & MOV Move to/from Special Registers](#)

答案范围: [MOV Move Data & MOV Move to/from Special Registers](#)

在以上两个页面中 (第二个页面，是只能在特权级使用的 MOV 指令)，由各自的 opcode - instruction 表可以总结出 MOV 指令的常见格式：

[opcode] [ModR/M] [SIB?] [disp?]	-- 最统一的编码模型
[opcode + reg no.] [imm]	-- 立即数到寄存器 (码)
[opcode] [addr]	-- 固定寄存器 (特殊编码)

第二张表的系统特权级指令：

[0F] [opcode] [ModR/M]

——本质上是用 0F 扩展 opcode 前缀，仍然用 ModR/M 编码寄存器。

(二) 必答题 2: 代码量统计

问: (1.1) shell 命令完成 PA1 的内容之后, nemu/目录下的所有.c 和.h 文件总共有多少行代码? (1.2) 你是使用什么命令得到这个结果的? (1.3) 和框架代码相比, 你在 PA1 中编写了多少行代码? (Hint: 目前 2017 分支中记录的正好是做 PA1 之前的状态, 思考一下应该如何回到"过去"?) (1.4) 你可以把这条命令写入 Makefile 中, 随着实验进度的推进, 你可以很方便地统计工程的代码行数, 例如敲入 `make count` 就会自动运行统计代码行数的命令。(2) 除去空行之外, nemu/目录下的所有.c 和.h 文件总共有多少行代码?

答:

首先回答怎么“回到过去”: 用 `git checkout` 命令。

```
1 git checkout [branch]
```

将 branch 替换为想要回到的过去状态。比如:

```
1 git checkout master
2 git checkout pa0
```

这样就可以恢复之前分支保存的代码状态。

接着回答 (1.1) (1.2) 和 (2) 的问题——使用什么命令得到 nemu/目录下所有.c 和.h 文件总代码行数, 以及怎么统计去除空行的代码行数。

可以通过 `git ls-files`:

```
1 git ls-files 'nemu/*.c' 'nemu/*.h' | xargs cat | wc -l
2 git ls-files 'nemu/*.c' 'nemu/*.h' | xargs grep -hEv '^[[:space:]]*$' | wc -l
```

分别能够统计总代码量和去除空行的代码量。得到的统计结果是:

pa0: 3487/2817

pa1: 4283/3506

(全代码/去空行)

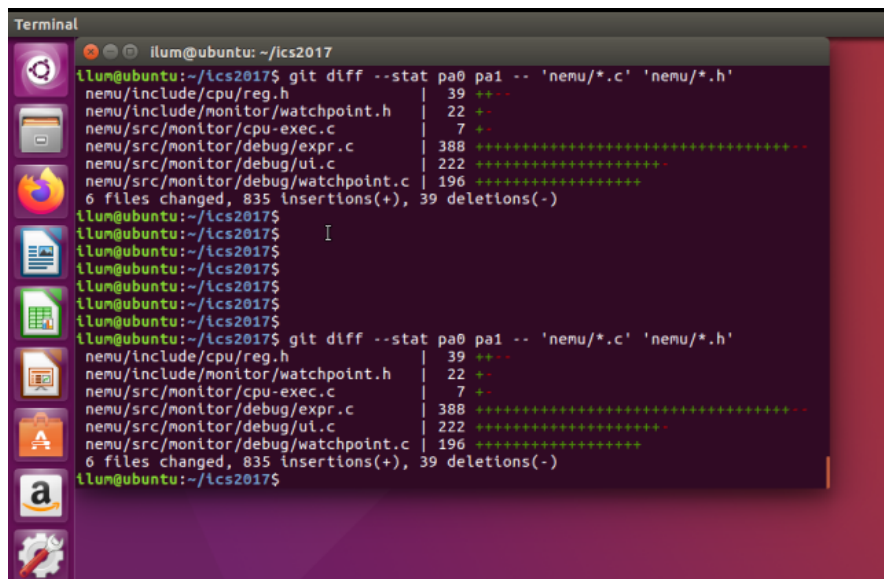
再者, 回答 (1.3) 的问题——和框架代码相比, 怎么得到新编写的代码行数。

可以通过 `git diff`, 能比较直观地展示增加代码情况:

```
1 git diff --stat pa0 pa1 -- nemu
```

或者换用 `--numstat` 选项, 没那么直观但更全面、更统计地显示“增加行数 / 删除行数 / 文件名”:

```
1 git diff --numstat pa0 pa1 -- nemu
```



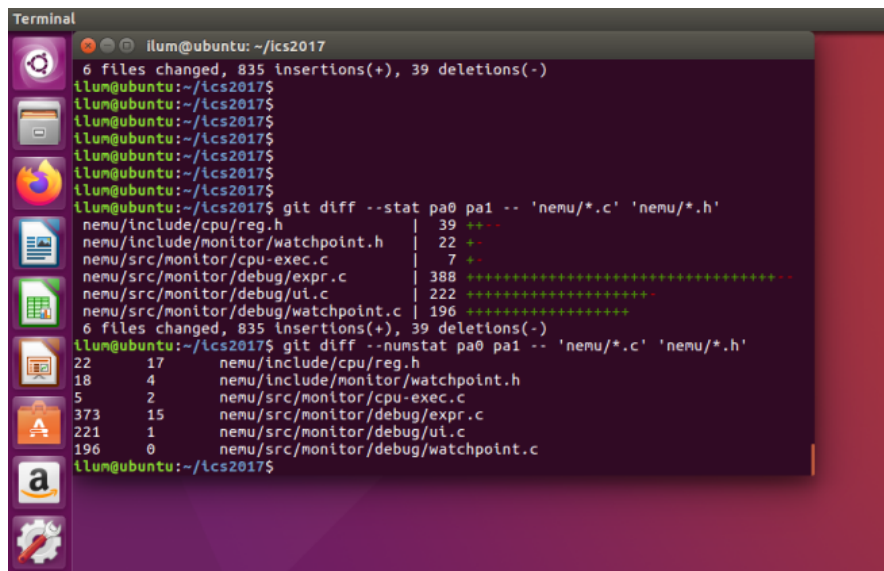
```

Terminal
illum@ubuntu: ~/ics2017
illum@ubuntu:~/ics2017$ git diff --stat pa0 pa1 -- 'nemu/*.c' 'nemu/*.h'
nemu/include/cpu/reg.h      | 39 +++-
nemu/include/monitor/watchpoint.h | 22 +-
nemu/src/monitor/cpu-exec.c | 7 +-
nemu/src/monitor/debug/expr.c | 388 ++++++
nemu/src/monitor/debug/ui.c  | 222 ++++++
nemu/src/monitor/debug/watchpoint.c | 196 ++++++
6 files changed, 835 insertions(+), 39 deletions(-)

illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$ git diff --stat pa0 pa1 -- 'nemu/*.c' 'nemu/*.h'
nemu/include/cpu/reg.h      | 39 +++-
nemu/include/monitor/watchpoint.h | 22 +-
nemu/src/monitor/cpu-exec.c | 7 +-
nemu/src/monitor/debug/expr.c | 388 ++++++
nemu/src/monitor/debug/ui.c  | 222 ++++++
nemu/src/monitor/debug/watchpoint.c | 196 ++++++
6 files changed, 835 insertions(+), 39 deletions(-)
illum@ubuntu:~/ics2017$

```

图 1: git diff -stat 结果



```

Terminal
illum@ubuntu: ~/ics2017
6 files changed, 835 insertions(+), 39 deletions(-)
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$
illum@ubuntu:~/ics2017$ git diff --stat pa0 pa1 -- 'nemu/*.c' 'nemu/*.h'
nemu/include/cpu/reg.h      | 39 +++-
nemu/include/monitor/watchpoint.h | 22 +-
nemu/src/monitor/cpu-exec.c | 7 +-
nemu/src/monitor/debug/expr.c | 388 ++++++
nemu/src/monitor/debug/ui.c  | 222 ++++++
nemu/src/monitor/debug/watchpoint.c | 196 ++++++
6 files changed, 835 insertions(+), 39 deletions(-)
illum@ubuntu:~/ics2017$ git diff --numstat pa0 pa1 -- 'nemu/*.c' 'nemu/*.h'
22 17      nemu/include/cpu/reg.h
18 4       nemu/include/monitor/watchpoint.h
5 2        nemu/src/monitor/cpu-exec.c
373 15     nemu/src/monitor/debug/expr.c
221 1      nemu/src/monitor/debug/ui.c
196 0      nemu/src/monitor/debug/watchpoint.c
illum@ubuntu:~/ics2017$

```

图 2: git diff -numstat 结果

得到分项数值后再附加命令去对增减代码行数进行统计：

```

1 git diff --numstat pa0 pa1 -- 'nemu/*.c' 'nemu/*.h' \
2 | awk '{ add += $1; del += $2 } END { printf "added: %d\ndeleted: %d\nnet: %d\n", add, del, add - del }'

```

命令的含义是，通过 \$n 取裸输出表中的数据，再用 += 累加每个文件的数据，而后分别输出增加行数、删减行数的统计值，以及两者的差值作为净增行数的统计值（直接进行简单的变量计算即可）。

```

Terminal Terminal File Edit View Search Terminal Help
illum@ubuntu: ~/lcs2017
6 files changed, 835 insertions(+), 39 deletions(-)
illum@ubuntu:~/lcs2017$ git diff --numstat pa0 pa1 -- 'nenu/*.c' 'nenu/*.h'
22      17      nenu/include/cpu/reg.h
18       4      nenu/include/monitor/watchpoint.h
5        2      nenu/src/monitor/cpu-exec.c
373     15      nenu/src/monitor/debug/expr.c
221      1      nenu/src/monitor/debug/ui.c
196      0      nenu/src/monitor/debug/watchpoint.c
illum@ubuntu:~/lcs2017$ git diff --numstat pa0 pa1 -- 'nenu/*.c' 'nenu/*.h' awk '
{ add += $1; del += $2 } END { printf "added: %d\ndeleted: %d\nnet: %d\n", add,
del, add - del }'
22      17      nenu/include/cpu/reg.h
18       4      nenu/include/monitor/watchpoint.h
5        2      nenu/src/monitor/cpu-exec.c
373     15      nenu/src/monitor/debug/expr.c
221      1      nenu/src/monitor/debug/ui.c
196      0      nenu/src/monitor/debug/watchpoint.c
illum@ubuntu:~/lcs2017$ git diff --numstat pa0 pa1 -- 'nenu/*.c' 'nenu/*.h' | awk
'{ add += $1; del += $2 } END { printf "added: %d\ndeleted: %d\nnet: %d\n", add
, del, add - del }'
added: 835
deleted: 39
net: 796
illum@ubuntu:~/lcs2017$

```

图 3: 统计结果

所以 PA1 相比原代码框架，新编写的代码行数：

835/796

(新加/净增)

下面回答问题 (1.4)，尝试将上面的命令整合到 Makefile 中。设计两个命令：

1. make count: 统计当前 nemu 目录下的 *.c/.h 代码总量，输出总代码行数、去除空行的总代码行数（使用 git ls-files 支持）。
2. make cmp [n] [m]: 比较 pa[n] 和 pa[m] 的代码，输出 pa[n] 相较于 pa[m] 的代码增减情况（使用 git diff --numstat 支持）。注：如果 [n] 缺省，默认到当前分支；如果 [m] 缺省，默认为 0，即跟 pa0 进行代码增减比较。

根目录 Makefile 新增代码如下：

```

COUNT_PATHS := 'nemu/*.c' 'nemu/*.h'
CMP_ARGS := $(filter-out cmp,$(MAKECMDGOALS))
ifneq ($(filter cmp,$(MAKECMDGOALS)),)
ifneq ($(word 1,$(CMP_ARGS)),)
n := $(word 1,$(CMP_ARGS))
endif
ifneq ($(word 2,$(CMP_ARGS)),)
m := $(word 2,$(CMP_ARGS))
endif
ifeq ($(origin n), undefined)
CURRENT_PA_N := $(shell git rev-parse --abbrev-ref HEAD 2>/dev/null |
sed -n 's/^pa\[0-9\][0-9]*\)$$/\1/p')
ifneq ($(CURRENT_PA_N),)
n := $(CURRENT_PA_N)

```

```

else
$(error 没有指名 n, 且当前分支名不是 pa数字; 请使用 make cmp <n> [m]
或 make cmp n=<n> [m=<m>])
endif
endif
endif
endif
m ?= 0

```

(中间代码略)

```

count:
    @echo "total lines:"
    @git ls-files -z $(COUNT_PATHS) | xargs -0 cat | wc -l
    @echo "nonblank lines:"
    @git ls-files -z $(COUNT_PATHS) | xargs -0 grep -hEv '^[[:space:]]'
    *$$' | wc -l

cmp:
    @echo "compare: pa$(n) relative to pa$(m)"
    @git diff --numstat pa$(m) pa$(n) -- $(COUNT_PATHS) | awk '{ add +=
    $$1; del += $$2 } END { printf "added: %d\ndeleted: %d\nnet: %d\n",
    add, del, add - del }'

```

在 nemu/Makefile 添加转发目标, 让 nemu 目录下也能使用 make cmp 和 make count:

```

TOP := $(shell git rev-parse --show-toplevel 2>/dev/null)
ifeq ($(strip $(TOP)),)
TOP := ..
endif
CMP_ARGS := $(filter-out cmp,$(MAKECMDGOALS))

```

(略)

```
.PHONY: app run submit clean count cmp
```

(略)

```

count:
    @$(MAKE) -C $(TOP) count

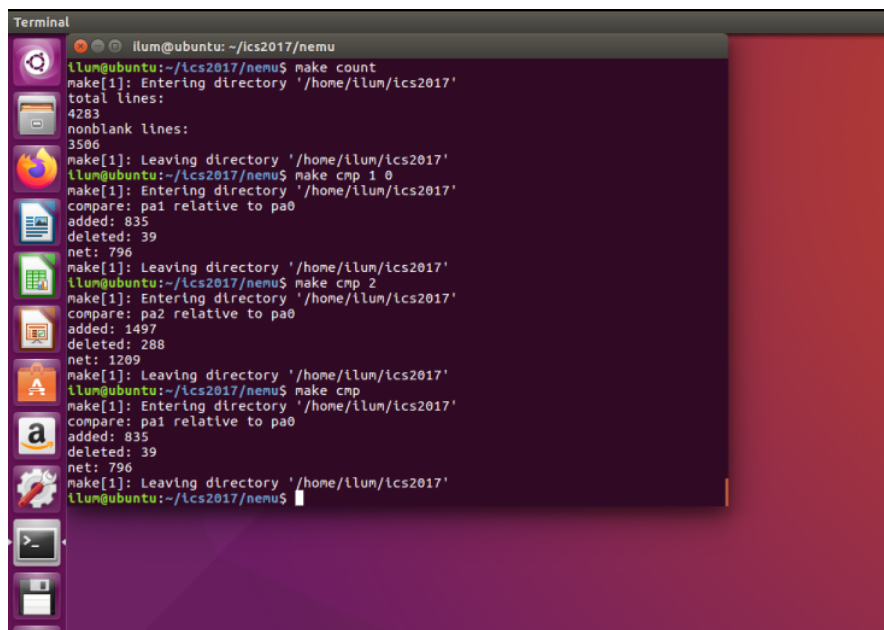
cmp:
    @$(MAKE) -C $(TOP) cmp $(CMP_ARGS)

```



```
ifneq ($(filter cmp,$(MAKECMDGOALS)),)
ifneq ($(CMP_ARGS),)
$(CMP_ARGS):
    @:
endif
endif
```

在 nemu/ 目录进行测试:



```
Terminal
ilum@ubuntu: ~/ics2017/nemu
ilum@ubuntu:~/ics2017/nemu$ make count
make[1]: Entering directory '/home/ilum/ics2017'
total lines:
4283
nonblank lines:
3506
make[1]: Leaving directory '/home/ilum/ics2017'
ilum@ubuntu:~/ics2017/nemu$ make cmp 1 0
make[1]: Entering directory '/home/ilum/ics2017'
compare: pa1 relative to pa0
added: 835
deleted: 39
net: 796
make[1]: Leaving directory '/home/ilum/ics2017'
ilum@ubuntu:~/ics2017/nemu$ make cmp 2
make[1]: Entering directory '/home/ilum/ics2017'
compare: pa2 relative to pa0
added: 1497
deleted: 288
net: 1209
make[1]: Leaving directory '/home/ilum/ics2017'
ilum@ubuntu:~/ics2017/nemu$ make cmp
make[1]: Entering directory '/home/ilum/ics2017'
compare: pa1 relative to pa0
added: 835
deleted: 39
net: 796
make[1]: Leaving directory '/home/ilum/ics2017'
ilum@ubuntu:~/ics2017/nemu$
```

图 4: make count & make cmp 测试结果

上面描述到的功能都能正常运行使用，结果是一致的。

(三) 必答题 3: gcc 编译选项

问: 使用 man 打开工程目录下的 Makefile 文件，你会在 CFLAGS 变量中看到 gcc 的一些编译选项。请解释 gcc 中的 -Wall 和 -Werror 有什么作用？为什么要使用 -Wall 和 -Werror？

答:

-Wall 和 -Werror 都是 gcc 的编译选项，主要和编译警告有关。

-Wall 的作用是开启一组常用的警告检查。它并非打开“所有”警告检查，而是打开 gcc 认为比较常见、有价值的一批。比如变量未使用、函数没有声明、某些可疑的类型转换、可能写错的表达式等。

-Werror 的作用是把警告当成错误报告。也就是说，原本只是警告、编译还能继续的情况，在加了 -Werror 后会直接导致编译失败（报 error: ...）。

在 PA 中，使用 -Wall 是为了让编译器检查更多可能存在问题的代码；使用 -Werror 是为了强制把这些可疑问题报告成错误，让实验者必须修掉，而不是带着 warning 继续写。这样可以 Let 代码更干净，避免一些隐藏 bug 在运行时才暴露。

二、 文内思考题

PA 在编程指导过程中提出的一些问题，单列出来进行系统的回答。

(一) Q: cpu_exec() 传参

问：在 cmd_c() 函数中，调用 cpu_exec() 的时候传入了参数-1，你知道这是什么意思吗？

答：

```
1 void cpu_exec(uint64_t n) {
```

cpu_exec 传入的整数实际上需要转化成无符号整数，而传入的 -1 是有符号的，实际中会被转换成一个最大的无符号正整数（18446744073709551615，即 $2^{64} - 1$ ）

```
1  . . . . .
2  for (; n > 0; n --) {
3      /* Execute one instruction, including instruction fetch,
4       * instruction decode, and the actual execution. */
5      exec_wrapper(print_flag);
6
7  . . . . .
8
9      if (nemu_state != NEMU_RUNNING) { return; }
10 }
11
12 . . . . .
```

cpu_exec 的逻辑是执行传入参数 n 这么多条指令，而当传入的是最大可支持的正整数，实际上不会真正跑这么多指令，通常会在程序结束、命中监视点/断点、或 nemu_state 被修改时提前返回。所以传入 -1 是一种用来表示“尽可能地、持续地、逐条指令运行下去，直到被打断”的调用写法（这里是对应 c / continue 命令）。

(二) Q: static 关键字的作用

问：框架代码中定义 wp_pool 等变量的时候使用了关键字 static，static 在此处的含义是什么？为什么要在此处使用它？

答：

```
1 static WP wp_pool[NR_WP];
2 static WP *head, *free_;
```

主要有两层含义：

1. **静态存储周期：**这些变量在程序运行期间一直存在，不会像局部变量那样函数返回后消失；监视点池是整个 NEMU 运行期间都需要维护的，所以相关变量必须长期保存。
2. **内部链接：**这些变量只在当前 watchpoint.c 内可见。其他源文件不能直接访问它们。

主要目的是对于监视点的管理集中封装在 watchpoint.c 内部，不因外部代码的运行而被影响甚至破坏（同时避免命名冲突等问题），使得其管理长期稳定安全且一致。

(三) Q: int3 相关问题

问: 我们知道 `int3` 指令不带任何操作数, 操作码为 1 个字节, 因此指令的长度是 1 个字节。这是必须的吗? 假设有一种 `x86` 体系结构的变种 `my-x86`, 除了 `int3` 指令的长度变成了 2 个字节之外, 其余指令和 `x86` 相同。在 `my-x86` 中, 文章中的断点机制还可以正常工作吗? 为什么?

答:

不是必须的。1 字节是专门用于软件断点的一种约定, 单字节便于插入任意指令位置且可恢复, 即能够可靠替换原指令并被 CPU 唯一识别为断点。

但如果变成 2 个字节别的不变, 就不能正常工作。因为断点要求能精确覆盖单字节位置, 如果改成 2 字节, 可能破坏当前指令或跨越到下一指令, 不再能可靠替换。

(四) Q: 断点的设置位置

问: 如果把断点设置在指令的非首字节 (中间或末尾), 会发生什么? 你可以在 GDB 中尝试一下, 然后思考并解释其中的缘由。

答:

理论上, 如果放在指令中间, 会破坏指令。GDB 的断点机制依赖指令边界, 如果不放在首字节取值时不会直接识别出来是断点, 反而取到的是被破坏了一字节的原指令, 会导致程序出错/崩溃。

(五) Q: 模拟器和调试器的区别

问: 模拟器 (Emulator) 和调试器 (Debugger) 有什么不同? 更具体地, 和 NEMU 相比, GDB 到底是如何调试程序的?

答:

Emulator 是用软件实现一整套虚拟 CPU, 亲自指导取值、译码、执行等操作, 是用软件模拟 CPU 硬件并自行指导其运作; Debugger 是在真实 CPU 上跑的程序基础上, 控制程序、查看状态、修改执行, 只是监视一个真正在真实 CPU 上跑的程序的状态。GDB 调试程序的能力来源于 OS 和 CPU 的异常机制, 核心在指令中插入 `int3` (`0xcc`) 也就是断点, 执行到此处会触发异常, OS 捕获异常后介入并把控制权移交给 GDB, GDB 接管后恢复原指令、调整 PC 然后在当前指令位置等待用户输入调试命令。所以 GDB 不亲自执行指令, 只是拦截执行过程使得用户有观察和进行调试的切入点。

三、 代码实现

本节按照手册指导顺序，根据 TODO 任务切分小节，逐步解析代码实现思路。

(一) TODO: 实现正确的寄存器结构体

首先需要修改寄存器结构使得 nemu 能够运行。

原始寄存器结构：

```
1 typedef struct {
2     struct {
3         uint32_t _32;
4         uint16_t _16;
5         uint8_t _8[2];
6     } gpr[8];
7
8     /* Do NOT change the order of the GPRs' definitions. */
9
10    /* In NEMU, rtlreg_t is exactly uint32_t. This makes RTL
11       instructions
12       * in PA2 able to directly access these registers.
13       */
14    rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
15
16    vaddr_t eip;
17 } CPU_state;
```

原始的寄存器设计实际上存了两套寄存器数据：

- gpr[8]：用于按 i386 编码访问（例如 gpr[3] 是 ebx）
- eax/ecx/...：用于直接按名字访问

这两套数据是“并列存在的”，不是同一份内存，理论上如果某一套当中的寄存器被改动，可能导致数据不一致。

同时考虑到 i386 寄存器的本质要求，数组形式是“编码视图”的。综合后，根据提示用 union 来改进实现：

```
1 typedef union {
2     rtlreg_t _32;
3     uint16_t _16;
4     uint8_t _8[2];
5 } GPR;
6
7 typedef struct {
8     union {
9         GPR gpr[8];
10
11     /* Do NOT change the order of the GPRs' definitions. */
12 }
```

```

13      /* In NEMU, rtlreg_t is exactly uint32_t. This makes RTL
        instructions
14      * in PA2 able to directly access these registers.
15      */
16      struct {
17          rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
18      };
19  };
20
21  vaddr_t eip;
22
23  } CPU_state;

```

这样使得寄存器只有一块一致性的内存，只是分为了两种视图（还是原来的索引/名字访问）。这样既节省了结构空间，又避免了同步问题，也保留了对原始的两访问需求的支持。

进行如上的代码修改之后，在 Ubuntu 终端运行 `make clean + make run`，已经能够成功启动 NEMU：

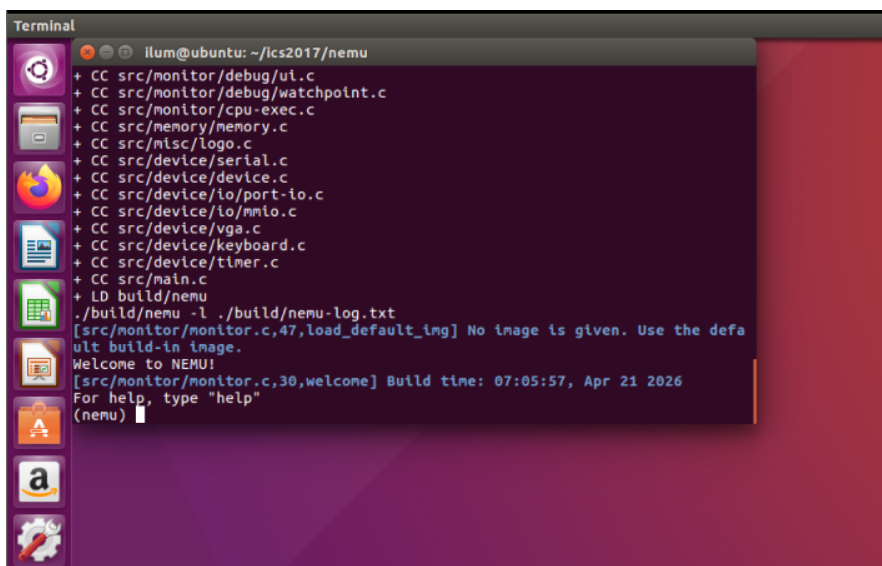


图 5: NEMU 能够运行

(二) TODO：添加单步执行命令 `si`

实现第一个命令——单步执行 `si` 命令。`si` 命令只允许有一个参数 `N`，代表执行的指令数目，如果缺省默认为 1。代码中使用 `args` 等进行基本的参数读取、检查、转换，然后调用 CPU 指令执行入口即可。用到了 `strtok()`（按指定分隔符切分成子串）、`strspn()`（计算字符串开头连续包含指定字符集合的长度）、`strlen()`（求长度）、`strtoull()`（转换成无符号长整型整数）等 `str` 函数。完整代码如下：

```

1  /* PA 1: {si [N]} - 单步执行 N 条指令；若 N 缺省，默认执行 1 条。 */
2  static int cmd_si(char *args) {
3      /* 缺省情况下执行 1 条指令。 */
4      uint64_t n = 1;
5

```

```

6  if (args != NULL) {
7      /* 解析 si 后继第一个参数，并检查是否还存在多余参数。 */
8      char *arg = strtok(args, " ");
9      char *extra = strtok(NULL, " ");
10
11     if (arg == NULL) {
12         cpu_exec(n);
13         return 0;
14     }
15
16     /* si 只接受一个正整数参数。 */
17     if (extra != NULL || strstr(arg, "0123456789") != strlen(arg)) {
18         printf("Usage: si [N]\n");
19         return 0;
20     }
21
22     n = strtoull(arg, NULL, 10);
23     /* 执行条数必须大于 0 */
24     if (n <= 0) {
25         printf("\33[1;31mcmd_error:\33[0m [N] should be greater than
26             0\n");
27         return 0;
28     }
29
30     /* 复用已有的执行入口，执行指定条数后返回到监视器。 */
31     cpu_exec(n);
32     return 0;
33 }

```

实现完成后进行测试，单步执行到程序停止：

```

Terminal
illum@ubuntu: ~/ics2017/nemu
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default b
uild-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 00:28:13, Apr 29 2026
For help, type "help"
(nemu) si 3
100000: b8 34 12 00 00          movl $0x1234,%eax
100005: b9 27 00 10 00          movl $0x100027,%ecx
10000a: 89 01                   movl %eax,(%ecx)
(nemu) si 2
10000c: 66 c7 41 04 01 00       movw $0x1,0x4(%ecx)
100012: bb 02 00 00 00          movl $0x2,%ebx
(nemu) si 1
100017: 66 c7 84 99 00 e0 ff 01 00 movw $0x1,-0x2000(%ecx,%ebx,4)
(nemu) si
100021: b8 00 00 00 00          movl $0x0,%eax
(nemu) si
nemu: HIT GOOD TRAP at eip = 0x00100026
100026: d6                   nemu trap (eax = 0)
(nemu) si
Program execution has ended. To restart the program, exit NEMU and run again.
(nemu)

```

图 6: 逐条指令运行至 GOOD TRAP

(三) TODO: 添加寄存器打印命令 info r

info 命令后跟的不是参数，而是子命令 r/w，不过识别、处理的逻辑和流程和处理一般的参数是基本一致的。代码如下：

```

1  /* PA 1: {info r} - 打印所有通用寄存器和 eip 的当前值。 */
2  static int cmd_info(char *args) {
3      if (args == NULL) {
4          printf("\33[1;31mcmd_error:\33[0m Unknown info
5              command\n\33[1;30minfo r - print registers\33[0m\n");
6          return 0;
7      }
8      /* 解析 info 的子命令，当前只支持 r。 */
9      char *subcmd = strtok(args, " ");
10     char *extra = strtok(NULL, " ");
11
12     if (extra != NULL) {
13         printf("\33[1;31mcmd_error:\33[0m Unknown info
14             command\n\33[1;30minfo r - print registers\33[0m\n");
15         return 0;
16     }
17     if (strcmp(subcmd, "r") == 0) {
18         int i;
19         /* 依次输出 8 个通用寄存器，再输出 eip。 */
20         for (i = 0; i < 8; i++) {
21             printf("%s\t0x%08x\n", reg_name(i, 4), reg_l(i));
22         }
23         printf("eip\t0x%08x\n", cpu.eip);
24         return 0;
25     }
26
27     printf("\33[1;31mcmd_error:\33[0m Unknown info command
28         '%s'\n\33[1;30minfo r - print registers\33[0m\n", subcmd);
29     return 0;
30 }

```

注意到，这里根据手册要求暂时只添加了 info r 用来打印寄存器状态，后续实现了监视点功能后，还需要添加 info w 用来打印监视点。

除此之外，代码中还尝试对于非法的命令（比如 info 后的子命令不存在）进行提示，但不直接终止 NEMU 的运行。

实现完成后，尝试运行 info r 命令：

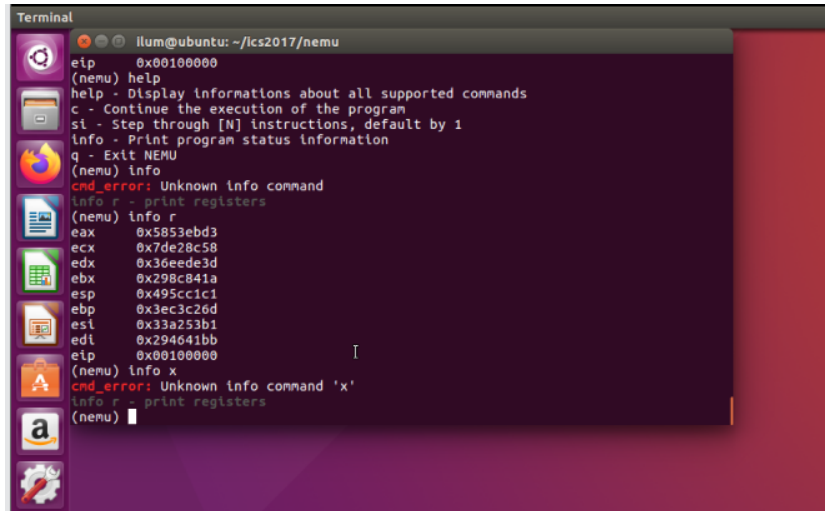


图 7: info r 运行结果（包括错误命令提示测试）

(四) TODO: 添加扫描内存命令 x N EXPR (简化版)

同样的，需要对输入的参数先进行解析和检查，逻辑基本相同，下面的代码就将这部分略去了：

```

1  /* PA 1: {x N EXPR} - 从十六进制起始地址开始，扫描并输出连续 N 个 4
   字节内存单元。 */
2  static int cmd_x(char *args) {
3      if (args == NULL) {
4          printf("\33[1;31mcmd_error:\33[0m Wrong x command
           format\n\33[1;30mx N 0xADDR - scan memory from hex
           address\33[0m\n");
5          return 0;
6      }
7
8      /* 解析 x 的两个参数：扫描个数 N 和十六进制起始地址 EXPR。 */
9      /* 此处代码略 ... */
10
11     /* 检查 N 必须是一个正整数，表示需要输出多少个 4 字节单元。 */
12     /* 此处代码略 ... */
13
14     /* ver.1: 简化版 EXPR 只允许写成 0x 开头的十六进制地址。 */
15     if (strlen(expr) <= 2 || expr[0] != '0' || (expr[1] != 'x' &&
           expr[1] != 'X')) {
16         strspn(expr + 2, "0123456789abcdefABCDEF") != strlen(expr +
           2)) {
17             printf("\33[1;31mcmd_error:\33[0m EXPR should be a hex address
                   like 0x100000\n");
18             return 0;
19         }
20         addr = strtoul(expr, NULL, 16);
21

```



```

22  /* 从起始地址开始，每次读取 4 字节并按十六进制输出。 */
23  for (i = 0; i < (int)n; i++) {
24      vaddr_t cur_addr = addr + i * 4;
25      uint32_t data = vaddr_read(cur_addr, 4);
26      printf("0x%08x: 0x%08x\n", cur_addr, data);
27  }
28
29  return 0;
30  }

```

保留的代码是其特有逻辑。指导手册要求的简化版 x 命令，只接受 0x 开头的十六进制地址。所以首先要对 EXPR 参数做检查，是否是十六进制；再把十六进制数转化成无符号整数。而后，转化得到的无符号整数作为起始地址，用 vaddr_read 基本功能函数，以 4 字节为单位扫描内存。实现后尝试扫描 10 个单位：

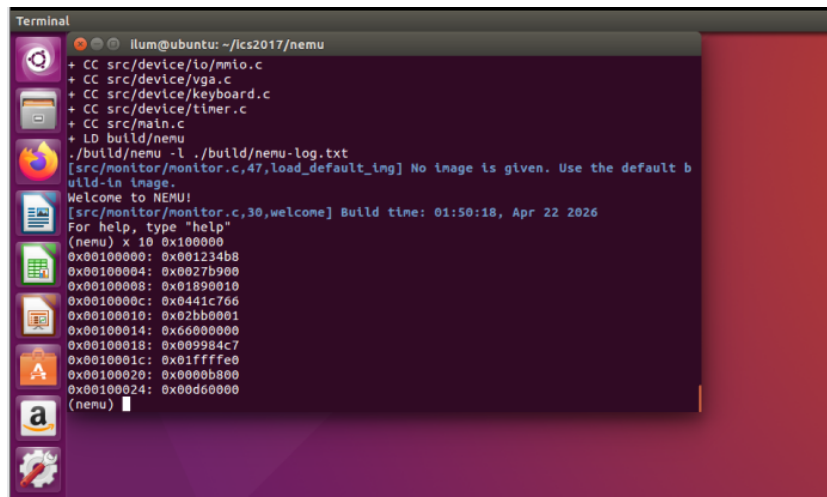


图 8: 用 x 命令扫描 10*4 Byte 内存

(五) CHECKPOINT: PA1 stage1

至此 PA1 stage1 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：

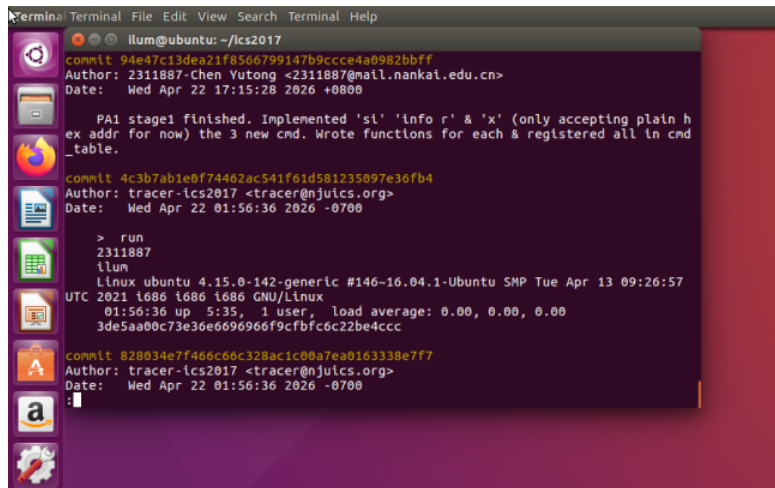


图 9: 手动 git 记录

(六) TODO: 算术表达式的词法分析

从这一节开始，进入词法分析的实现。这一部分只实现对于算术表达式的词法分析。

首先补足 Token 类型，添加 TK_NUM:

```
1 enum {
2     TK_NOTYPE = 256, TK_EQ, TK_NUM
3
4     /* TODO: Add more token types */
5
6 };
```

并同步地添加新的匹配 rules，涵盖所有四则运算符和十进制整数，以及括号：

```
1 static struct rule {
2     char *regex;
3     int token_type;
4 } rules[] = {
5
6     /* TODO: Add more rules.
7      * Pay attention to the precedence level of different rules.
8      */
9
10    {"[0-9]+", TK_NUM},    // 十进制整数
11    {" +", TK_NOTYPE},    // 空格串
12    {"\\+", '+'},        // 加号
13    {"-", '-'},          // 减号
14    {"\\*", '*'},        // 乘号
15    {"/", '/'},          // 除号
16    {"\\(", '('},        // 左括号
17    {"\\)", ')'},        // 右括号
18    {"==", TK_EQ}        // 相等比较
19 };
```

在 `make_token` 函数中实现 `token` 的匹配, 和匹配后对其信息的记录。根据实验手册的要求, 空白符不当作 `token`, 不记录任何信息直接跳过; 而其他 `token`, 都必须记录类型和词素两种信息, 必须完整。同时, 遵从实验 KISS 原则, 对于溢出 (`token` 数组溢出、`token` 过长) 异常直接用 `assert(0)` 终止, 降低代码 debug 代价、保证能够验证基本逻辑的正确, 后续再进行处理。其核心代码如下:

```

1  /* ... */
2  while (e[position] != '\0') {
3      /* Try all rules one by one. */
4      /* ... */
5
6      switch (rules[i].token_type) {
7          case TK_NOTYPE:
8              /* 空白符不需要记录成 token, 直接跳过。 */
9              break;
10         case TK_NUM:
11             case '+':
12             case '-':
13             case '*':
14             case '/':
15             case '(':
16             case ')':
17         case TK_EQ:
18             /* 实验要求保持 KISS, token 数组溢出时直接终止。 */
19             if (nr_token >= (int)(sizeof(tokens) /
20                 sizeof(tokens[0]))) {
21                 assert(0);
22             }
23             /* 无论什么类型的 token, 只要入表就要保存对应子串。 */
24             if (substr_len >= (int)sizeof(tokens[nr_token].str)) {
25                 assert(0);
26             }
27             /* 记录 token 类型, 并把当前匹配到的词素写入 str。 */
28             tokens[nr_token].type = rules[i].token_type;
29             strncpy(tokens[nr_token].str, substr_start, substr_len);
30             tokens[nr_token].str[substr_len] = '\0';
31             nr_token++;
32             break;
33         default:
34             TODO();
35     }
36     break;
37 }
38 }
39
40 if (i == NR_REGEX) {
41     printf("no match at position %d\n%s\n%.s^\n", position, e,

```

```

        position, "");
42     return false;
43 }
44 }
45 /* ... */

```

(七) TODO: 实现算数表达式递归求值 (含负号)

递归求值的函数构造完全按照实验指导手册的建议实施。下面分块简单说明：

```

1  /* PA 1: 分治递归地求解 p..q 对应的 token 子表达式。 */
2  static uint32_t eval(int p, int q, bool *success) {
3      /* ... some variables ... */
4      /* ... */
5
6      if (p > q) {
7          printf("bad expression: empty subexpression\n");
8          *success = false;
9          return 0;
10     }

```

使用指导手册的方法，将 token 左右分别标记 p 和 q。首先判断 p>q，也就是 token 为空的情况，先排除这种错误情形。

```

1     if (p == q) {
2         /* 单个 token 的表达式当前只可能是一个十进制整数。 */
3         if (tokens[p].type != TK_NUM) {
4             printf("bad expression: token '%s' is not a number\n",
5                 tokens[p].str);
6             *success = false;
7             return 0;
8         }
9
10        return strtoul(tokens[p].str, NULL, 10);

```

接着先处理递归最深一层的情况，也就是分裂到只有单个十进制整数的时候。这时候进行简单的合法判断，然后将字符串转化成无符号整数返回即可。

```

1     if (check_parentheses(p, q, &bad) == true) {
2         /* 若整个表达式被一对最外层括号完整包围，则直接去掉这层括号。 */
3         return eval(p + 1, q - 1, success);
4     }

```

去括号。优先级应当在处理多 token 串中最高，所以应该先写。其中 check_parentheses() 函数单独编写，如下：

```

1  /* PA 1: 求值辅助函数：
2   * 检查 p..q 对应的子表达式是否被一对最外层括号完整包围。
3   * 如果括号本身不匹配，则通过 bad 标记表达式非法。

```

```
4  */
5  static bool check_parentheses(int p, int q, bool *bad) {
6      int i;
7      int balance = 0;
8
9      *bad = false;
10
11     if (p > q) {
12         *bad = true;
13         return false;
14     }
15
16     /* 先检查整个区间中的括号是否匹配。 */
17     for (i = p; i <= q; i++) {
18         if (tokens[i].type == '(') {
19             balance++;
20         }
21         else if (tokens[i].type == ')') {
22             balance--;
23             if (balance < 0) {
24                 *bad = true;
25                 return false;
26             }
27         }
28     }
29
30     if (balance != 0) {
31         *bad = true;
32         return false;
33     }
34
35     /* 若首尾不是一对括号，则该表达式不属于 "(" <expr> ")" 形式。 */
36     if (tokens[p].type != '(' || tokens[q].type != ')') {
37         return false;
38     }
39
40     balance = 0;
41
42     /* 若最外层左括号在 q 之前就已经闭合，
43        则说明整个表达式并未被单独包围。 */
44     for (i = p; i <= q; i++) {
45         if (tokens[i].type == '(') {
46             balance++;
47         }
48         else if (tokens[i].type == ')') {
49             balance--;
50         }
```

```

51     if (balance == 0 && i < q) {
52         return false;
53     }
54 }
55
56 return true;
57 }

```

也是完全按照手册思路展开来写的，不再赘述分析其逻辑构成。

```

1  /* 否则寻找当前表达式最后一步执行的 dominant operator。 */
2  op = find_dominant_operator(p, q);
3  if (op < 0) {
4      printf("bad expression: cannot find dominant operator\n");
5      *success = false;
6      return 0;
7  }

```

如果不是单个 token，就要在 token 串里面找 dominant op（当前式子里面最后被计算/优先级最低的运算符）了。其中 find_dominant_operator() 代码如下：

```

1  /* PA 1: 求值辅助函数：
2   * 在 p..q 中寻找 dominant operator，
3   * 只考虑括号外层的运算符。
4   * 对二元四则运算符，同优先级下取最右边，以符合左结合。
5   * 对单目负号，同优先级下取最左边，以符合右结合。
6   */
7  static int find_dominant_operator(int p, int q) {
8      int i;
9      int op = -1;
10     int min_precedence = 0x7fffffff;
11     int balance = 0;
12
13     for (i = p; i <= q; i++) {
14         int cur_precedence;
15
16         if (tokens[i].type == '(') {
17             balance++;
18             continue;
19         }
20
21         if (tokens[i].type == ')') {
22             balance--;
23             continue;
24         }
25
26         if (balance > 0) {
27             continue;
28         }
29

```

```

30     cur_precedence = get_precedence(tokens[i].type);
31     if (cur_precedence == 0) {
32         continue;
33     }
34
35     if (cur_precedence < min_precedence) {
36         min_precedence = cur_precedence;
37         op = i;
38         continue;
39     }
40
41     if (cur_precedence == min_precedence) {
42         if (tokens[i].type == TK_NEG) {
43             /* 单目负号右结合，应保留最左边那个作为当前层的 dominant
44              * operator。 */
45             continue;
46         }
47
48         /* 二元运算左结合，同优先级下更新为更右边的运算符。 */
49         op = i;
50     }
51     return op;
52 }

```

这个函数是纯自编逻辑的。主要存在的问题是多个同优先级的运算符怎么处理：对二元四则运算符，同优先级下取最右边，以符合左结合；对单目负号，同优先级下取最左边，以符合右结合。其他只是简单的优先级不同的符号之间的比较，较为简单。

```

1     /* 单目负号只需要递归求右侧的子表达式。 */
2     if (tokens[op].type == TK_NEG) {
3         val2 = eval(op + 1, q, success);
4         if (*success == false) {
5             return 0;
6         }
7         return 0 - val2;
8     }

```

目前的单目运算符只有一个“-”负号，递归求右侧子表达式就行。

```

1     val1 = eval(p, op - 1, success);
2     if (*success == false) {
3         return 0;
4     }
5
6     val2 = eval(op + 1, q, success);
7     if (*success == false) {
8         return 0;
9     }
10

```

```
11  /* 分别求出左右子表达式后，再按 dominant operator 合并结果。 */
12  switch (tokens[op].type) {
13      case '+':
14          return val1 + val2;
15      case '-':
16          return val1 - val2;
17      case '*':
18          return val1 * val2;
19      case '/':
20          if (val2 == 0) {
21              printf("bad expression: divide by zero\n");
22              *success = false;
23              return 0;
24          }
25          return val1 / val2;
26      default:
27          printf("bad expression: unsupported operator '%s'\n",
28                  tokens[op].str);
29          *success = false;
30          return 0;
31  }
```

对于二元运算符，先各自递归地求 dominant op 左右两边的值，再按照 dominant op 的实际意义进行四则运算合并结果即可。

当命令中涉及到求值任务时，预计会调用 `expr()` 函数进行表达式求值。

```
1  uint32_t expr(char *e, bool *success) {
2      if (!make_token(e)) {
3          *success = false;
4          return 0;
5      }
6
7      /* 空表达式不合法，直接返回失败。 */
8      if (nr_token == 0) {
9          printf("bad expression: empty expression\n");
10         *success = false;
11         return 0;
12     }
13
14     /* 先统一识别出所有单目负号，再进入递归求值。 */
15     preprocess_tokens();
16
17     /* 从整个 token 区间开始递归求值。 */
18     *success = true;
19     return eval(0, nr_token - 1, success);
20 }
```

可以看到调用了 `eval()` 函数进行求值（目前还只是纯算术表达式）。

指导书特别提到了对于负号（“-”）的特殊处理，需要当作前缀单目符号处理。经过思考后，决

定采用这样的策略: 如果某个“-”不扮演“二元减号”的中介角色(某一层的 dominant operator), 那么就把它解释成单目负号, 并允许连续叠加 (1)。

比如, $1 + -1$ 会识别为“ $1 + (-1)$ ”, $1 - - - -1$ 则会识别为“ $1 - (-1)$ ”。负号的识别应当在正式递归运算之前就排查清楚, 所以特地写了 `preprocess_tokens()` 函数做此处理, 可以看到在上面 `expr()` 函数体中已经被调用。代码:

```
1  /* PA 1: 求值辅助函数:
2   * 词法分析结束后, 根据上下文区分减号和单目负号。
3   * 若 '-' 出现在表达式开头, 或出现在另一个运算符/左括号之后,
4   * 则它不可能是二元减号, 应解释成单目负号 TK_NEG。
5   */
6  static void preprocess_tokens(void) {
7      int i;
8
9      for (i = 0; i < nr_token; i++) {
10         if (tokens[i].type != '-') {
11             continue;
12         }
13
14         if (i == 0 || !is_value_token(tokens[i - 1].type)) {
15             tokens[i].type = TK_NEG;
16         }
17     }
18 }
```

关于负号识别与处理, 指导书提出的两个问题:

- 负号和减号都是-, 如何区分它们?
- 负号是个单目运算符, 分裂的时候需要注意什么?

这里作简答:

- 负号和减号怎么区分: 看上下文。“-”前面如果是数字或“(”, 它就是二元减号; 否则就是单目负号 TK_NEG.
- 单目负号分裂时注意什么: 它不是二元运算符, 所以不能分成左右两棵子树, 只能递归求它右边那个子表达式, 然后返回 $0 - val$.

这其中还涉及到其他一些较为简单的辅助功能函数, 比如用来给算符优先级排序的函数 `get_precedence()`, 用来判断子表达式结尾的函数 `is_value_token()`, 其实现思路都较为简单不再展示。

(八) TODO: 实现更完整的表达式求值

参照指导书提供的 BNF:

```
<expr> ::= <decimal-number>
          | <hexadecimal-number>      # 以"0x"开头的
```

```

| <reg_name>                # 以"$"开头的
| "(" <expr> ")"
| <expr> "+" <expr>
| <expr> "-" <expr>
| <expr> "*" <expr>
| <expr> "/" <expr>
| <expr> "==" <expr>
| <expr> "!=" <expr>
| <expr> "&&" <expr>
| <expr> "||" <expr>
| "!" <expr>
| "<" <expr>

```

首先补全 token 类型和相应的 rule:

```

1  enum {
2      TK_NOTYPE = 256,
3      TK_EQ,
4      TK_NEQ,
5      TK_AND,
6      TK_OR,
7      TK_NUM,
8      TK_HEX,
9      TK_REG,
10     TK_NEG,
11     TK_DEREF,
12     TK_NOT
13
14     /* TODO: Add more token types */
15 };
16
17 static struct rule {
18     char *regex;
19     int token_type;
20 } rules[] = {
21
22     /* TODO: Add more rules.
23      * Pay attention to the precedence level of different rules.
24      */
25
26     {" +", TK_NOTYPE},           // 空格串
27     {"0[xX][0-9a-fA-F]+", TK_HEX}, // 十六进制整数
28     {"[0-9]+", TK_NUM},          // 十进制整数
29     {"\\$[a-zA-Z][a-zA-Z0-9]*", TK_REG}, // 寄存器名
30     {"==", TK_EQ},               // 相等比较
31     {"!=", TK_NEQ},              // 不等比较
32     {"&&", TK_AND},              // 逻辑与

```

```

33 { "\\|\\|", TK_OR},           // 逻辑或
34 {"!", TK_NOT},               // 逻辑非
35 {"\\+", '+'},                // 加号
36 {"-", '-'},                  // 减号或负号
37 {"\\*", '*'},                // 乘号或解引用
38 {"/", '/'},                  // 除号
39 {"\\(", '('},                // 左括号
40 {"\\)", ')'},                // 右括号
41 };

```

在正式求值之前的预处理阶段，在之前负号判断的基础上再加上对于单目解引用符号“*”的识别：

```

1  /* PA 1: 求值辅助函数：
2   * 词法分析结束后，根据上下文区分二元和单目运算符。
3   * '-' 在表达式开头，或前一个 token
4   *   不是“值结尾”时，解释成单目负号。
5   * '*' 在表达式开头，或前一个 token 不是“值结尾”时，解释成解引用。
6   */
7  static void preprocess_tokens(void) {
8      int i;
9
10     for (i = 0; i < nr_token; i++) {
11         if (tokens[i].type == '-') {
12             if (i == 0 || !is_value_token(tokens[i - 1].type)) {
13                 tokens[i].type = TK_NEG;
14             }
15         }
16         else if (tokens[i].type == '*') {
17             if (i == 0 || !is_value_token(tokens[i - 1].type)) {
18                 tokens[i].type = TK_DEREF;
19             }
20         }
21     }
22 }

```

加入新运算符以后，eval() 只需要在相应的地方添加对新运算符的识别和具体语义的运算即可，基本是复制之前已经实现的运算符的逻辑，不再展示。不过，由于现在加入了寄存器等其他取值来源，复杂度升高，所以考虑用 eval_single_token() 函数代替原来算数表达式中最小子表达式直接取值的逻辑，其中包含了完整的读取 token 值的逻辑：

```

1  /* PA 1: 辅助函数：读取单个字面量或寄存器 token 的值。 */
2  static uint32_t eval_single_token(int p, bool *success) {
3      switch (tokens[p].type) {
4          case TK_NUM:
5              return strtoul(tokens[p].str, NULL, 10);
6          case TK_HEX:
7              return strtoul(tokens[p].str, NULL, 16);
8          case TK_REG:
9              return read_register(tokens[p].str, success);

```

```

10     default:
11         printf("bad expression: token '%s' is not a value\n",
12               tokens[p].str);
13         *success = false;
14         return 0;
15     }
16 }

```

其中 `read_register()` 函数也是新编写的，大致逻辑是按照寄存器名遍历寄存器群去找到相应的寄存器，代码逻辑比较简单不展示。

总得来说，`expr()` 函数不改变，只是原先的 `preprocess_tokens()` 和 `eval()` 函数有所变化，并在其中调用了新的功能封装函数。

(九) TODO: 实现 p 和 x 命令

在 `expr.c` 中完整实现了表达式计算以后，就可以在 `ui.c` 中针对表达式求值命令 `p` 命令和之前未完整实现的扫描内存命令 `x` 命令进行 debug 的用户接口代码实现。

p 命令函数 `cmd_p()`:

```

1  /* PA 1: {p EXPR} - 对表达式 EXPR 求值并输出结果。 */
2  static int cmd_p(char *args) {
3      bool success = true;
4      uint32_t val;
5
6      args = skip_spaces(args);
7      if (args == NULL || *args == '\0') {
8          printf("\33[1;31mcmd_error:\33[0m Wrong p command
9              format\n\33[1;30mp EXPR - evaluate expression\33[0m\n");
10         return 0;
11     }
12
13     /* 调用表达式求值入口，支持当前 expr.c 已实现的所有表达式类型。 */
14     val = expr(args, &success);
15     if (!success) {
16         printf("\33[1;31mcmd_error:\33[0m Bad expression\n");
17         return 0;
18     }
19
20     /* 输出十进制结果，同时输出十六进制结果，便于调试。 */
21     printf("%u (0x%08x)\n", val, val);
22     return 0;
23 }

```

简单地直接调用 `expr()` 即可。

之前由于未实现 `EXPR` 的完整求值流程，`x` 命令中的表达式只支持人工输入的十六进制字面常量（`0x` 开头的 16 进制数）。现在在 `EXPR` 完整实现之后，也可以直接调用 `expr()` 进行表达式计算：

```

1  /* PA 1: {x N EXPR} - 先对 EXPR 求值，再从该地址开始扫描连续 N 个 4
2     字节单元。 */

```

```

2 static int cmd_x(char *args) {
3     /* some simple processing, basically no change ... */
4
5     /* x 的第一个参数是扫描个数 N，其余整串都作为表达式 EXPR 传给
        expr(). */
6     n_str = strtok(args, " ");
7     uint64_t n;
8     vaddr_t addr;
9     int i;
10    bool success = true;
11
12    if (n_str == NULL) {
13        printf("\33[1;31mcmd_error:\33[0m Wrong x command
            format\n\33[1;30m N EXPR - scan memory from evaluated
            address\33[0m\n");
14        return 0;
15    }
16
17    expr_str = n_str + strlen(n_str) + 1;
18    expr_str = skip_spaces(expr_str);
19    if (*expr_str == '\0') {
20        printf("\33[1;31mcmd_error:\33[0m Wrong x command
            format\n\33[1;30m N EXPR - scan memory from evaluated
            address\33[0m\n");
21        return 0;
22    }
23
24    /* no change ... */
25
26    /* 由 expr() 统一求 EXPR 的值，
        可兼容寄存器、十六进制、逻辑/算术表达式等写法。 */
27    addr = expr(expr_str, &success);
28    if (!success) {
29        printf("\33[1;31mcmd_error:\33[0m Bad expression\n");
30        return 0;
31    }
32
33    /* no change ... */
34
35    return 0;
36 }

```

省略的部分和原来第一版实现基本一致，为节省空间不再展示。现在尝试 make clean + make run 之后分别调用 p 命令和 x 命令检验实现效果：

1. 最基本的常量运算

```

Terminal
ilum@ubuntu: ~/ics2017/nemu
+ CC src/device/serial.c
+ CC src/device/device.c
+ CC src/device/io/port-io.c
+ CC src/device/io/mmio.c
+ CC src/device/vga.c
+ CC src/device/keyboard.c
+ CC src/device/timer.c
+ CC src/main.c
+ LD build/nemu
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default b
uild-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 18:04:56, Apr 24 2026
For help, type "help"
(nemu) p 1+2
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
0 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "+" at position 1 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
2 with len 1: 2
3 (0x00000003)
(nemu)

```

图 10: 一次加法

```

Terminal
ilum@ubuntu: ~/ics2017/nemu
[src/monitor/debug/expr.c,410,make_token] match rules[14] = "\" at position 9 w
ith len 1: )
0 (0x00000000)
(nemu) p 1+2-3*4/6
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
0 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "+" at position 1 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
2 with len 1: 2
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-" at position 3 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
4 with len 1: 3
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "*" at position 5 w
ith len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
6 with len 1: 4
[src/monitor/debug/expr.c,410,make_token] match rules[12] = "/" at position 7 wi
th len 1: /
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
8 with len 1: 6
1 (0x00000001)
(nemu)

```

图 11: 多运算符四则运算

2. 位运算

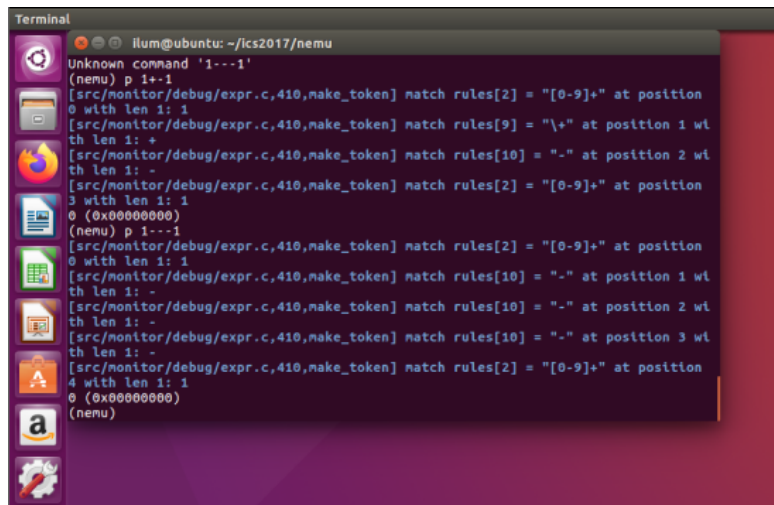
```

Terminal
ilum@ubuntu: ~/ics2017/nemu
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
4 with len 1: 3
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "\" at position 5 w
ith len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
6 with len 1: 4
[src/monitor/debug/expr.c,410,make_token] match rules[12] = "/" at position 7 wi
th len 1: /
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
8 with len 1: 6
1 (0x00000001)
(nemu) p 6|3&&5
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
0 with len 1: 6
[src/monitor/debug/expr.c,410,make_token] match rules[7] = "||" at position 1
with len 2: ||
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
3 with len 1: 3
[src/monitor/debug/expr.c,410,make_token] match rules[6] = "&&" at position 4 wi
th len 2: &&
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
6 with len 1: 5
1 (0x00000001)
(nemu)

```

图 12: 位运算

3. 单目负号



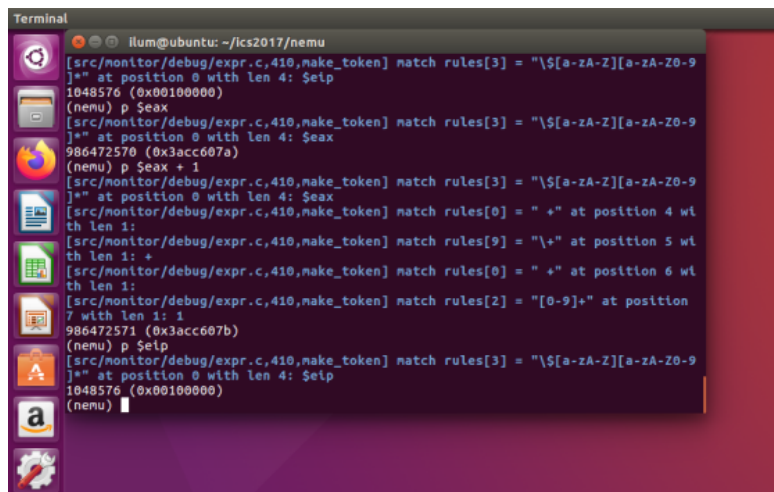
```

Terminal
ilum@ubuntu: ~/ics2017/nemu
Unknown command '1--1'
(nemu) p 1--1
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
0 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "\"+" at position 1 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-" at position 2 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
3 with len 1: 1
0 (0x00000000)
(nemu) p 1--1
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
0 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-" at position 1 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-" at position 2 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-" at position 3 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
4 with len 1: 1
0 (0x00000000)
(nemu)

```

图 13: 负号

4. 寄存器



```

Terminal
ilum@ubuntu: ~/ics2017/nemu
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "\\$[a-zA-Z][a-zA-Z0-9
]*" at position 0 with len 4: $eax
1048576 (0x00100000)
(nemu) p $eax
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "\\$[a-zA-Z][a-zA-Z0-9
]*" at position 0 with len 4: $eax
986472570 (0x3acc607a)
(nemu) p $eax + 1
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "\\$[a-zA-Z][a-zA-Z0-9
]*" at position 0 with len 4: $eax
[src/monitor/debug/expr.c,410,make_token] match rules[0] = "+" at position 4 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "\"+" at position 5 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[0] = "+" at position 6 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
7 with len 1: 1
986472571 (0x3acc607b)
(nemu) p $eax
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "\\$[a-zA-Z][a-zA-Z0-9
]*" at position 0 with len 4: $eax
1048576 (0x00100000)
(nemu)

```

图 14: 寄存器

5. 十六进制运算和解引用

```

Terminal
ilum@ubuntu: ~/ics2017/nemu
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xx][0-9a-fA-F]+" a
t position 2 with len 7: 0x12345
[src/monitor/debug/expr.c,410,make_token] match rules[14] = "\"\" at position 9 w
ith len 1: )
0 (0x00000000)
(nemu) p 0x12345+100
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xx][0-9a-fA-F]+" a
t position 0 with len 7: 0x12345
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "\"\" at position 7 w
ith len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
8 with len 3: 100
74665 (0x000123a9)
(nemu) p *(0x12345)
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "\"\" at position 0 w
ith len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[13] = "\"\" at position 1 w
ith len 1: (
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xx][0-9a-fA-F]+" a
t position 2 with len 7: 0x12345
[src/monitor/debug/expr.c,410,make_token] match rules[14] = "\"\" at position 9 w
ith len 1: )
0 (0x00000000)
(nemu)

```

图 15: 十六进制运算和解引用

6. 负数

```

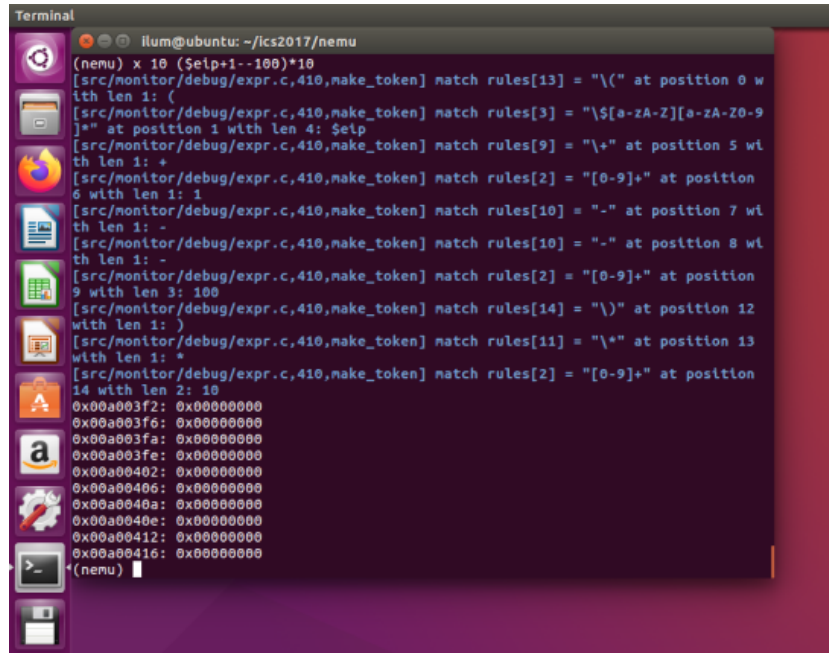
Terminal
ilum@ubuntu: ~/ics2017/nemu
Unknown command '(1+2-3*4)/6'
(nemu) p (1+2-3*4)/6
[src/monitor/debug/expr.c,410,make_token] match rules[13] = "\"\" at position 0 w
ith len 1: (
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
1 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "\"\" at position 2 w
ith len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
3 with len 1: 2
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "-\" at position 4 w
ith len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
5 with len 1: 3
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "\"\" at position 6 w
ith len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
7 with len 1: 4
[src/monitor/debug/expr.c,410,make_token] match rules[14] = "\"\" at position 8 w
ith len 1: )
[src/monitor/debug/expr.c,410,make_token] match rules[12] = "\"\" at position 9 w
ith len 1: /
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "[0-9]+" at position
10 with len 1: 6
715827881 (0x2aaaaaa9)
(nemu) p -1

```

图 16: 计算结果为负数

说明：负数向下溢出成大整数（取模 2^{32} ）。但指导手册要求所有数据都是 `uint32_t` 类型，所以这个现象是正常的。

最后再对 `x` 命令做测试：



```

Terminal
illum@ubuntu: ~/ics2017/nemu
(nemu) x 10 ($elp+1--100)*10
[src/monitor/debug/expr.c,410,make_token] match rules[13] = "\"(\" at position 0 w
ith len 1: (
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "\"$[a-zA-Z][a-zA-Z0-9
]*\" at position 1 with len 4: $elp
[src/monitor/debug/expr.c,410,make_token] match rules[9] = "\"+\" at position 5 wi
th len 1: +
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "\"[0-9]+\" at position
6 with len 1: 1
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "\"-\" at position 7 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[10] = "\"-\" at position 8 wi
th len 1: -
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "\"[0-9]+\" at position
9 with len 3: 100
[src/monitor/debug/expr.c,410,make_token] match rules[14] = "\"}\" at position 12
with len 1: )
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "\"*\" at position 13
with len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[2] = "\"[0-9]+\" at position
14 with len 2: 10
0x00a003f2: 0x00000000
0x00a003f6: 0x00000000
0x00a003fa: 0x00000000
0x00a003fe: 0x00000000
0x00a00402: 0x00000000
0x00a00406: 0x00000000
0x00a0040a: 0x00000000
0x00a0040e: 0x00000000
0x00a00412: 0x00000000
0x00a00416: 0x00000000
(nemu)

```

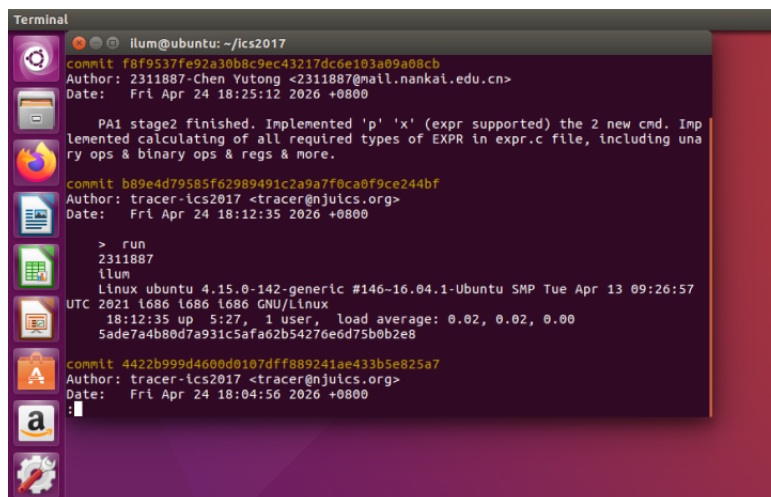
图 17: x 命令扫描内存结果

运算输出结果都是正确、合理的。并且每个 token 匹配都有 Log 输出（原代码框架即包含的功能）。

(十) CHECKPOINT: PA1 stage2

至此 PA1 stage1 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：



```

Terminal
illum@ubuntu: ~/ics2017
commit f8f9537fe92a30b8c9ec43217dc6e103a09a08cb
Author: 2311887-Chen Yutong <2311887@mail.nankai.edu.cn>
Date: Fri Apr 24 18:25:12 2020 +0800

    PA1 stage2 finished. Implemented 'p' 'x' (expr supported) the 2 new cmd. Imp
    lemented calculating of all required types of EXPR in expr.c file, including una
    ry ops & binary ops & regs & more.

commit b89e4d79585f62989491c2a9a7f0ca0f9ce244bf
Author: tracer-ics2017 <tracer@njuics.org>
Date: Fri Apr 24 18:12:35 2020 +0800

    > run
    2311887
    illum
    Linux ubuntu 4.15.0-142-generic #146-16.04.1-Ubuntu SMP Tue Apr 13 09:26:57
    UTC 2021 i686 i686 i686 GNU/Linux
    18:12:35 up 5:27, 1 user, load average: 0.02, 0.02, 0.00
    Sade7a4b80d7a931c5afa62b54276e6d75bb02e8

commit 4422b999d4600d0107dfff889241ae433b5e825a7
Author: tracer-ics2017 <tracer@njuics.org>
Date: Fri Apr 24 18:04:56 2020 +0800

```

图 18: 手动 git 记录

(十一) TODO: 实现监视点池的管理

监视点池本质上是一个链表。监视点的操作主要就是两个：获取和释放。代码原框架已经提供了一个空闲链表和一个使用中链表，那么获取和释放对应的链表操作就显而易见了。

对于监视点的获取, 要从空闲链表头部取出一个节点, 并将新申请的节点挂到使用中链表头部:

```
1 // PA 1: 获取一个空闲监视点
2 WP* new_wp() {
3     // 没有空闲监视点时直接中止, 防止后续解引用空指针
4     assert(free_ != NULL);
5
6     // 从空闲链表头部取出一个节点
7     WP *wp = free_;
8     free_ = free_->next;
9
10    // 将新申请的节点挂到使用中链表头部, 便于后续统一遍历检查
11    wp->next = head;
12    head = wp;
13
14    return wp;
15 }
```

而对于监视点的释放/回收, 要在使用中链表中遍历地查找到所指定的监视点节点, 将其从使用中链表卸下, 再挂回到空闲链表头部:

```
1 // PA 1: 获取一个空闲监视点
2 WP* new_wp() {
3     // 没有空闲监视点时直接中止, 防止后续解引用空指针
4     assert(free_ != NULL);
5
6     // 从空闲链表头部取出一个节点
7     WP *wp = free_;
8     free_ = free_->next;
9
10    // 将新申请的节点挂到使用中链表头部, 便于后续统一遍历检查
11    wp->next = head;
12    head = wp;
13
14    return wp;
15 }
16
17 // PA 1: 回收一个使用中的监视点
18 void free_wp(WP *wp) {
19     assert(wp != NULL);
20
21    // 在使用中链表中找到 wp, 同时记录它前一个节点以便卸下
22    WP *prev = NULL;
23    WP *cur = head;
24    while (cur != NULL && cur != wp) {
25        prev = cur;
26        cur = cur->next;
27    }
28 }
```

```

29 // 找不到说明 wp 不是正在使用的监视点，给出错误信息后直接返回
30 if (cur == NULL) {
31     printf("\33[1;31mwatchpoint_error:\33[0m watchpoint %#d is not
        in use\n", wp->NO);
32     return;
33 }
34
35 // 从使用中链表摘下 wp
36 if (prev == NULL) {
37     head = wp->next;
38 }
39 else {
40     prev->next = wp->next;
41 }
42
43 // 将 wp 归还到空闲链表头部，之后可再次被 new_wp() 分配
44 wp->next = free_;
45 free_ = wp;
46 }

```

在代码上，链表的挂载和卸下，主要体现在 next 的更新，有时涉及到 head 的切换。代码框架的链表很简单，其具体代码操作不再细致分析。

(十二) TODO: 实现监视点实现监视点设置命令 w 删除命令 d

为保持代码的封装性，编写了一套专门面向外部代码调用的监视点功能函数。

第一个函数是 set_watchpoint(), 用来设置一个监视点。前半部分进行输入参数式（代表添加的监视点所要监视的变量，一旦该变量的值发生变化就触发监视点）的检查和转换，而后调用 expr() 对参数式进行求值，最后调用 new_wp() 申请新的空闲监视点：

```

1 // PA 1: 设置一个新的监视点，保存表达式 EXPR 和它当前的值
2 bool set_watchpoint(char *expr_str) {
3     /* 参数检查与转换过程，略 */
4     /* ... */
5
6     char saved_expr[WP_EXPR_LEN];
7     memcpy(saved_expr, expr_str, len);
8     saved_expr[len] = '\0';
9
10    // expr() 接收可写字符串，这里使用副本求值，
        避免破坏保存下来的表达式文本
11    char eval_expr[WP_EXPR_LEN];
12    strcpy(eval_expr, saved_expr);
13
14    bool success = true;
15    uint32_t val = expr(eval_expr, &success);
16    if (!success) {
17        printf("\33[1;31mwatchpoint_error:\33[0m Bad expression\n");
18        return false;

```

```

19 }
20
21 // 申请 watchpoint
22 WP *wp = new_wp();
23 strcpy(wp->expr, saved_expr);
24 wp->last_value = val;
25
26 printf("Watchpoint %d: %s = %u (0x%08x)\n", wp->NO, wp->expr,
        val, val);
27 return true;
28 }

```

set_watchpoint() 被 w 命令使用。在 ui.c 中新写 cmd_w() 函数用于实现 w 命令逻辑，格式是 w EXPR，函数中将用户输入的表达式直接传入上面的接口函数即可：

```

1 /* PA 1: {w EXPR} - 设置监视点，当 EXPR 的值变化时暂停执行。 */
2 static int cmd_w(char *args) {
3     args = skip_spaces(args);
4     if (args == NULL || *args == '\0') {
5         printf("\33[1;31mcmd_error:\33[0m Wrong w command
        format\n\33[1;30mw EXPR - set watchpoint\33[0m\n");
6         return 0;
7     }
8
9     set_watchpoint(args);
10    return 0;
11 }

```

第二个函数是 delete_watchpoint()，用来删除/释放一个监视点。这里需要在使用中链表检查是否存在此序号的监视点。如果检查通过调用 free_wp() 释放监视点即可：

```

1 // PA 1: 按序号删除监视点
2 bool delete_watchpoint(int no) {
3     WP *wp = find_wp(no);
4     if (wp == NULL) {
5         printf("\33[1;31mwatchpoint_error:\33[0m Watchpoint %d not
        found\n", no);
6         return false;
7     }
8
9     free_wp(wp);
10    printf("Deleted watchpoint %d\n", no);
11    return true;
12 }

```

delete_watchpoint() 被 d 命令使用。新写 cmd_d() 函数用于实现 d 命令逻辑，格式是 d N，函数中要先对正整数参数 N 有效性做检查，而后直接传参给上面的接口函数即可：

```

1 /* PA 1: {d N} - 删除序号为 N 的监视点。 */
2 static int cmd_d(char *args) {

```

```

3  args = skip_spaces(args);
4  if (args == NULL || *args == '\0') {
5      printf("\33[1;31mcmd_error:\33[0m Wrong d command
        format\n\33[1;30md N - delete watchpoint N\33[0m\n");
6      return 0;
7  }
8
9  char *no_str = strtok(args, " ");
10 char *extra = strtok(NULL, " ");
11 if (no_str == NULL || extra != NULL || strspn(no_str,
        "0123456789") != strlen(no_str)) {
12     printf("\33[1;31mcmd_error:\33[0m Wrong d command
        format\n\33[1;30md N - delete watchpoint N\33[0m\n");
13     return 0;
14 }
15
16 uint64_t no = strtoull(no_str, NULL, 10);
17 if (no > 0xffffffff) {
18     printf("\33[1;31mcmd_error:\33[0m Watchpoint number is too
        large\n");
19     return 0;
20 }
21
22 delete_watchpoint((int)no);
23 return 0;
24 }

```

第三个函数是 `info_watchpoints()`，用于打印所有使用中的监视点。打印信息要完备，目前考虑到的关键信息有：序号、所监视的 EXPR（申请时输入）、当前值。其中当前值同时打印十进制和十六进制，方便 debug。代码如下：

```

1  // PA 1: 打印当前使用中的监视点信息
2  void info_watchpoints() {
3      WP *wp;
4      if (head == NULL) {
5          printf("No watchpoints.\n");
6          return;
7      }
8
9      printf("Num      Type      Disp Enb Expr
        Value\n");
10     for (wp = head; wp != NULL; wp = wp->next) {
11         printf("%-7d %-14s %-4s %-3s %-20s %u (0x%08x)\n",
12             wp->NO, "watchpoint", "keep", "y", wp->expr,
13             wp->last_value, wp->last_value);
14     }
15 }

```

`info_watchpoints()` 被 `info w` 命令所使用。之前的 `info` 命令只实现了打印寄存器状态

的 info r, 现在在 cmd_info() 函数中再增加 info w 相关的逻辑分支, 检查参数后直接交给上面的接口函数打印即可:

```

1  /* PA 1: {info r/info w} - 打印寄存器或监视点信息。 */
2  static int cmd_info(char *args) {
3      /* checking, no change ... */
4
5      /* 解析 info 的子命令, 当前支持 r 和 w。 */
6      char *subcmd = strtok(args, " ");
7      char *extra = strtok(NULL, " ");
8
9      if (extra != NULL) {
10         printf("\33[1;31mcmd_error:\33[0m Unknown info
            command\n\33[1;30minfo r - print registers\ninfo w - print
            watchpoints\33[0m\n");
11         return 0;
12     }
13
14     if (strcmp(subcmd, "r") == 0) {
15         /* info r, no change ... */
16     }
17
18     if (strcmp(subcmd, "w") == 0) {
19         /* 打印所有正在使用的监视点。 */
20         info_watchpoints();
21         return 0;
22     }
23
24     printf("\33[1;31mcmd_error:\33[0m Unknown info command
            '%s'\n\33[1;30minfo r - print registers\ninfo w - print
            watchpoints\33[0m\n", subcmd);
25     return 0;
26 }

```

最后, 还编写一个 check_watchpoints() 函数, 用于检查所有监视点的触发条件, 在其监视的值发生变化时暂停 NEMU. 使用 for 循环逐一检查所有使用中的监视点, 只要有一个监视点所监视的值发生变化就会引发暂停; 如果有多个监视点同时引发暂停, 会将它们的信息全部打印出来并暂停:

```

1  // PA 1: 每执行完一条指令后检查所有监视点, 任意一个变化都暂停 NEMU
2  bool check_watchpoints() {
3      bool triggered = false;
4      WP *wp;
5
6      for (wp = head; wp != NULL; wp = wp->next) {
7          bool success = true;
8          /* 求值和检查, 略 */
9
10         // 打印被触发的监视点信息

```

```

11     if (new_value != wp->last_value) {
12         printf("\33[1;31mWatchpoint %d triggered:\33[0m %s\n",
                wp->NO, wp->expr);
13         printf("Old value = %u (0x%08x)\n", wp->last_value,
                wp->last_value);
14         printf("New value = %u (0x%08x)\n", new_value, new_value);
15
16         // 触发后更新保存值，避免继续运行时因同一次变化反复停下
17         wp->last_value = new_value;
18         triggered = true;
19     }
20 }
21
22 if (triggered) {
23     // 可能同一条指令触发多个监视点，上面会全部打印后再统一暂停
24     nemu_state = NEMU_STOP;
25 }
26
27 return triggered;
28 }

```

check_watchpoints() 在 cpu_exec.c 的 CPU 命令执行主函数 cpu_exec() 中被调用。主函数中会执行下面的分支逻辑：如果在 DEBUG 模式下，每执行一条指令就检查有没有任一监视点对应的表达式值发生了变化，如有，立即切换到 NEMU_STOP 暂停执行（后面的检查和切换逻辑都在上面的接口函数中具体实现，cpu_exec() 只需要调用接口）。

```

1  /* ... */
2  #ifdef DEBUG
3      /* PA 1: 每条指令执行完后检查监视点，若表达式值变化则暂停执行 */
4      if (nemu_state == NEMU_RUNNING) {
5          check_watchpoints();
6      }
7  #endif

```

手册提到要考虑一些监视点的特殊情况，下面是本代码实现采用的处理策略：

- 如果设置监视点时表达式为空、非法或过长，则打印错误，不申请监视点。
- 申请监视点时没有空闲监视点，new_wp() 保持 assert(free_ != NULL) 终止 NEMU。
- 删除不存在的监视点，只会打印报错，不终止 NEMU。
- 如果同一条指令触发多个监视点，会一次性全部打印出来，然后暂停 NEMU。

上面代码编写完成后编译并测试监视点相关指令，先通过 w EXPR 设置监视点，再检查 info w 所有监视点状况，最后 d N 删除断点（这里展示的是报错）：

```

Terminal
illum@ubuntu: ~/ics2017/nemu
+ CC src/device/io/mmio.c
+ CC src/device/vga.c
+ CC src/device/keyboard.c
+ CC src/device/timer.c
+ CC src/main.c
+ LD build/nemu
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:34:42, Apr 24 2026
For help, type "help"
(nemu) w *0x2000
[src/monitor/debug/expr.c,410,make_token] match rules[11] = "*" at position 0 with len 1: *
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 1 with len 6: 0x2000
Watchpoint 0: *0x2000 = 0 (0x00000000)
(nemu) info w
Num      Type      Disp Enb What      Value
0      watchpoint    keep y  *0x2000      0 (0x00000000)
(nemu) d 1
watchpoint_error: Watchpoint 1 not found
(nemu)

```

图 19: 监视点测试

暂时没有指令执行无法排查特殊情况的处理效果。

下面是一个监视点当做断点用的例子：

```

Terminal
illum@ubuntu: ~/ics2017/nemu
illum@ubuntu:~/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:34:42, Apr 24 2026
For help, type "help"
(nemu) p $elp
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "[a-zA-Z][a-zA-Z0-9]" at position 0 with len 4: $elp
1048576 (0x00100000)
(nemu) w $elp==0x0010000a
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "[a-zA-Z][a-zA-Z0-9]" at position 0 with len 4: $elp
[src/monitor/debug/expr.c,410,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 10: 0x0010000a
Watchpoint 0: $elp==0x0010000a = 0 (0x00000000)
(nemu) c
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "[a-zA-Z][a-zA-Z0-9]" at position 0 with len 4: $elp
[src/monitor/debug/expr.c,410,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 10: 0x0010000a
[src/monitor/debug/expr.c,410,make_token] match rules[3] = "[a-zA-Z][a-zA-Z0-9]" at position 0 with len 4: $elp
[src/monitor/debug/expr.c,410,make_token] match rules[4] = "==" at position 4 with len 2: ==
[src/monitor/debug/expr.c,410,make_token] match rules[1] = "0[xX][0-9a-fA-F]+" at position 6 with len 10: 0x0010000a
Watchpoint 0 triggered: $elp==0x0010000a
Old value = 0 (0x00000000)
New value = 1 (0x00000001)
(nemu)

```

图 20: 断点测试

(十三) CHECKPOINT: PA1 stage3

至此 PA1 stage3 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：



```
Terminal
ilum@ubuntu: ~/ics2017
commit a117bb98cf750f50e6a7b69430654daf5d1da158
Author: 2311887-Chen Yutong <2311887@mail.nankai.edu.cn>
Date:   Fri Apr 24 23:05:22 2026 +0800

    PA1 stage3 finished. PA1 all stages cleared. Implemented 'w' 'd' & 'info w'
    the 3 new cmd. Implemented watchpoint function in watchpoint.h/.c.

commit 1c4f9dc1bf333556aa765341b3e58d39ca9c4d7b
Author: tracer-ics2017 <tracer@njuics.org>
Date:   Fri Apr 24 22:25:00 2026 +0800

    > run
    2311887
    ilum
    Linux ubuntu 4.15.0-142-generic #146-16.04.1-Ubuntu SMP Tue Apr 13 09:26:57
    UTC 2021 i686 i686 i686 GNU/Linux
    22:25:00 up 9:22, 1 user, load average: 0.00, 0.00, 0.00
    3ca3d85e64070d4832acb927e55769928ffb7f32

commit b750de111b93d215a7164e4ef46a2c6731969de7
Author: tracer-ics2017 <tracer@njuics.org>
Date:   Fri Apr 24 22:24:26 2026 +0800
:
```

图 21: 手动 git 记录

至此 PA 1 代码实现全部完成

四、 实验环境说明

由于整个课程的实验部分没有要求提交 PA0 / 环境配置一节的报告，此处对实验环境的搭建作简单说明。

(一) 虚拟机配置

采用 VMware 15.5.0 + Ubuntu 16.04 ([ubuntu-16.04.4-desktop-i386.iso](#)) 有可视化界面的虚拟机环境。

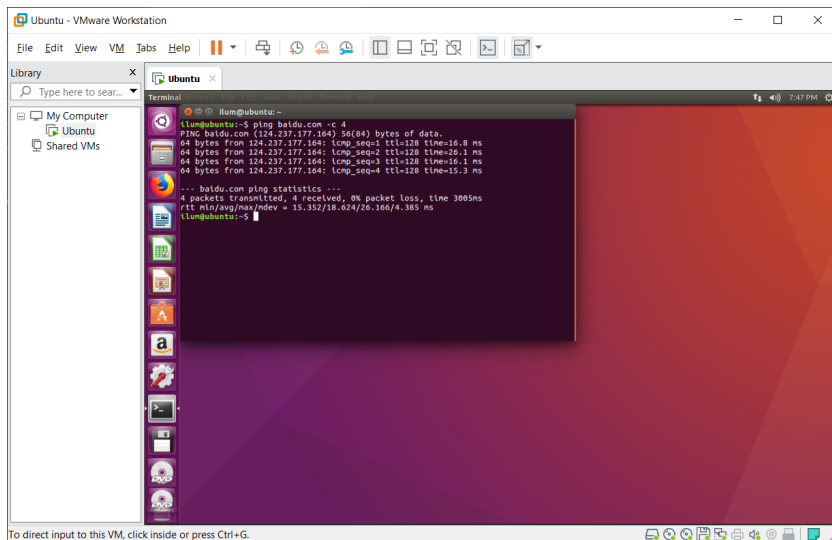


图 22: VMWare + Ubuntu

网络连接无问题。

按照指导书要求，使用 `sudo apt-get install` 命令下载并安装相应的资源工具包。

(二) IDE 配置

此前编程习惯为 vscode 的 IDE 环境，故尝试将此 Ubuntu 虚拟机连接到 vscode。

此前尝试 vscode 的 SSH 功能、VMware 自带的共享文件夹，均失败。

最终使用 SSHFS 的方案，把 Ubuntu 的目录通过 SSHFS 作为一个附加卷挂载到宿主 Windows。安装 WinFsp 和 SSHFS-Win 工具，而后只需要在 Windows cmd 输入命令 `net use X: \\sshfs\[username]@[ip.addr]\[password] /user:[username]`（方括号需要替换为相应内容）在宿主机的文件系统根目录形成一个新的卷轴“X:”，其中内容就是对 Ubuntu 虚拟机指定文件夹的挂载。在 X 盘内右键用 vscode 打开即可用 IDE 环境来编辑代码，同步生效于 Ubuntu 虚拟机。

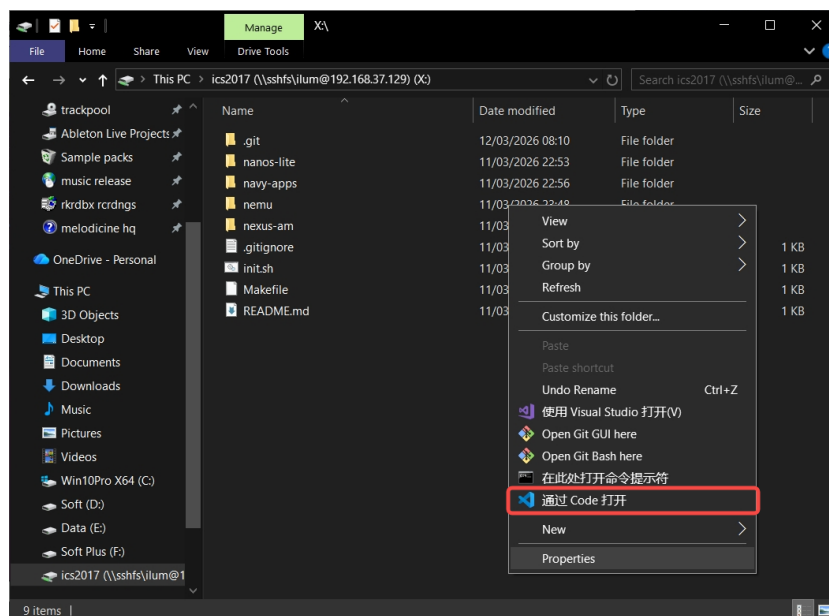


图 23: 本地 vscode 编写代码

五、 总结

这是第一次 PA 实验，在实验手册的指导下，初识了 NEMU 这个“CPU/计算机”模拟器。

PA1 的代码实现部分主要是对 NEMU 的 debug 系统进行了补全、完善，在实现过程中温习并强化了有关不限于 union、str 函数、链表等方面的知识，同时对于面向对象编程、函数接口封装等编程思想有了新体会。对于指导手册强调的 KISS (Keep It Simple, Stupid) 原则深有体会，并在多个命令的实现过程中始终遵守着 KISS 原则，最先保证基础逻辑的正确，不急于将庞大的 NEMU 体系一步实现到位。

在必答题、思考题中，从问题本身涉及到的知识点来说，学到了有关模拟器调试器、GCC 编译器的知识，并且复习了 C 语言相关、体系结构硬件方面的知识。而从解决问题的过程和手段来说，学到了一些新的 git 命令，也学会了如何查阅形如 i386 manual 这样体系庞大的用户手册，对于今后工程作业，掌握这些基本的工具是有帮助的。

PA1 实验代码已同步备份至 GitHub 仓库。